

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA1223

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A APPLICATION BASED QUESTIONS

Code of Conduct:

I Mogana Priya (192321165) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.


Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes

77.5%
48

of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	3	5	2	3	2	15
Q2	3	5	3	4	2	17
Q3	2	5	3	3	1.5	14.5
Q4	4	4	3	3	2	16
Q5	3	4	3	3	2	15

17.5

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements ⁴	Good understanding with minor gaps ³	Basic understanding; some requirements unclear ²	Poor understanding or misinterpretation of the problem ¹
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems ⁶	Applies relevant concepts correctly with minor errors ⁵	Applies basic concepts with limited accuracy ⁴	Fails to apply appropriate concepts ¹⁻²
Problem-Solving Approach	20	Logical, well-structured, and efficient approach ⁴	Mostly logical approach with minor gaps ³	Basic approach with some inconsistencies ²	Illogical or unclear approach ¹
Accuracy of Solution	20	Completely correct solution with proper justification ⁴	Mostly correct with minor errors ³	Partially correct solution ²	Incorrect or incomplete solution ¹
Clarity	10	Clear, well-organized, and properly explained solution ²	Understandable with minor clarity issues ^{1.5}	Limited clarity and explanation ¹	Poorly presented and difficult to understand ⁰

QUESTION 1

Computer performance depends on the coordinated operation of the CPU, memory, and I/O subsystem. The CPU executes instructions, memory stores instructions and data, and I/O devices transfer information between the computer and external devices. When the amount of data transfer increases, delays may occur because of memory latency, I/O waiting time, and bus contention.

Interaction Between CPU, Memory and I/O

1. The CPU executes instructions and performs arithmetic, logical, and control operations.
2. Main memory stores program instructions and data required by the CPU.
3. I/O devices transfer data between the computer and external devices.
4. The CPU, memory, and I/O devices communicate through address, data, and control buses.
5. When the CPU requires data that is not immediately available, it may have to wait for memory or I/O operations.
6. Frequent memory accesses increase memory stall time and reduce CPU efficiency.
7. Continuous I/O operations increase bus traffic and may result in bus contention.
8. If the incoming data rate is higher than the processing rate, data may accumulate in buffers and increase response time.
9. Cache memory reduces memory-access delays by storing frequently used instructions and data close to the CPU.
10. Direct Memory Access (DMA) allows large blocks of data to be transferred between memory and I/O devices with minimal CPU involvement.

Single-Bus Structure

A single-bus structure uses one common communication path between the CPU, memory, and I/O devices.

Advantages:

1. It has a simple architecture.
2. It requires less hardware.
3. It has lower implementation cost.
4. Its control mechanism is comparatively simple.

Limitations:

1. CPU, memory, and I/O devices compete for the same bus.
2. Only limited transfers can occur at a time.
3. Bus contention increases when several components request the bus simultaneously.
4. The shared bus can become a bottleneck in high-speed systems.

Cache stores frequently accessed instructions and data closer to the CPU, reducing memory-access latency.

3. Branch Prediction

Branch prediction reduces delays caused by conditional control-flow instructions.

4. Superscalar Execution

Superscalar processors can issue multiple instructions during the same clock cycle when sufficient execution resources are available.

5. Out-of-Order Execution

Independent instructions can execute while another instruction is waiting for data.

6. Multicore Architecture

Multiple processing cores can execute independent workloads concurrently.

7. Direct Memory Access

DMA allows large data transfers between memory and I/O devices without requiring continuous CPU involvement.

Role of Benchmarking

Benchmarking is the process of measuring system performance using defined workloads.

Important performance parameters include:

1. Execution time
2. Instruction count
3. CPI
4. Clock cycle time
5. Memory latency
6. Cache hit rate
7. Throughput
8. CPU utilization

The relationship between these parameters can be expressed as:

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

Benchmarking can help identify whether performance limitations are caused by processor execution, memory access, or data-transfer operations.

Error Detection and Minimization

Conversion and transmission errors can be detected or minimized using:

1. Parity checking

Multi-Bus Structure

A multi-bus structure provides multiple communication paths between system components.

Advantages:

1. It supports greater parallel data transfer.
2. It reduces bus contention.
3. It provides higher data-transfer bandwidth.
4. CPU and memory operations can occur with less interference from I/O operations.
5. It improves overall system throughput.

Limitation:

The main disadvantage is increased hardware complexity and implementation cost.

Improvements in Instruction Execution

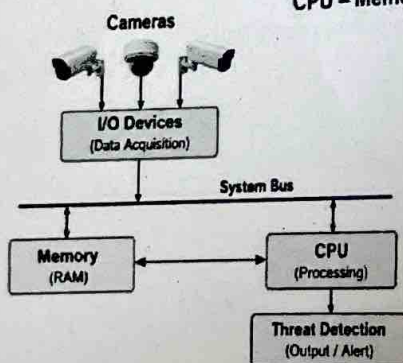
1. Instruction pipelining overlaps instruction fetch, decode, and execution stages.
2. Cache memory reduces the average time required to access instructions and data.
3. Branch prediction reduces delays caused by conditional branch instructions.
4. Out-of-order execution allows independent instructions to execute while another instruction is waiting.
5. Superscalar processors can execute multiple instructions during the same clock cycle.
6. Multicore processors allow different tasks to be processed simultaneously.
7. DMA reduces CPU overhead during large data transfers.

Conclusion

The performance of a computer system depends on efficient communication between the CPU, memory, and I/O devices. A single-bus structure is simple but can suffer from bus contention, whereas a multi-bus structure provides better parallelism and bandwidth. Cache memory, DMA, pipelining, branch prediction, and parallel processing can significantly improve overall system performance.

Smart City Surveillance System

CPU - Memory - I/O Interaction



- Cameras send data to I/O Devices
- I/O Devices transfer data over System Bus
- CPU processes the data using instructions stored in Memory
- Output is generated for threat detection and alerts

QUESTION 2

The fetch-decode-execute cycle is the fundamental process through which a processor executes instructions. CPU performance depends not only on clock frequency but also on the number of clock cycles required to execute each instruction. This is represented by Cycles Per Instruction (CPI).

Fetch Stage

1. The Program Counter (PC) contains the address of the next instruction.
2. The address is sent to the memory system.
3. The instruction is fetched from cache or main memory.
4. The fetched instruction is placed into the Instruction Register (IR).
5. The Program Counter is updated to identify the next instruction.

Decode Stage

1. The control unit examines the opcode of the instruction.
2. It determines the operation that needs to be performed.
3. The required registers and operands are identified.
4. Appropriate control signals are generated.
5. The processor determines the execution unit required for the instruction.

Execute Stage

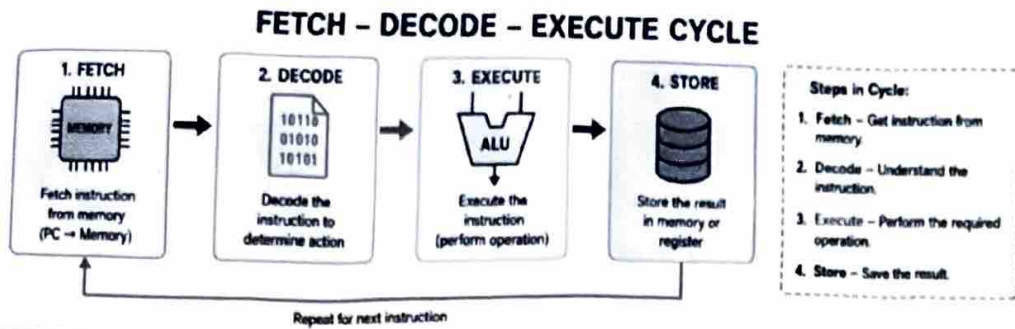
1. The required operation is performed by the appropriate functional unit.
2. Arithmetic and logical operations are performed by the Arithmetic Logic Unit (ALU).
3. Memory instructions access the required data.
4. Branch instructions may modify the Program Counter.
5. The result is written into the destination register or memory.

Effect of CPI on Execution Time

CPU execution time can be represented as:

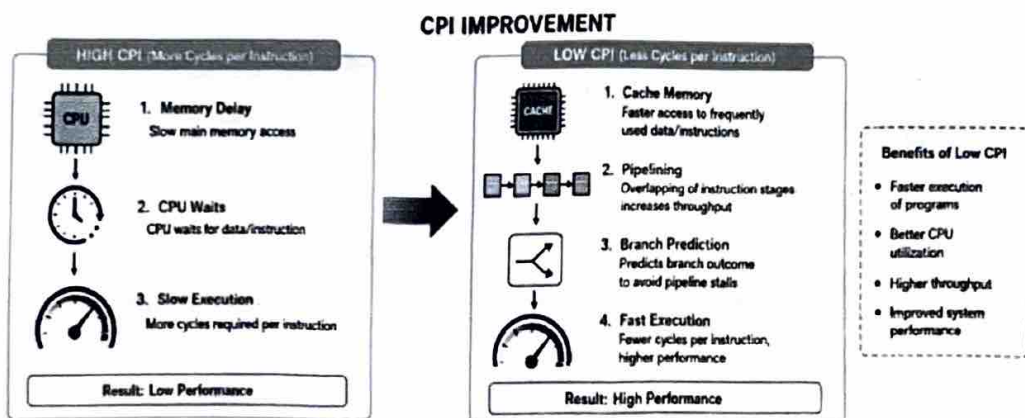
$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

Therefore, a high CPI means that more clock cycles are required for each instruction, increasing total execution time.



Causes of High CPI

1. Cache misses require data to be retrieved from slower levels of memory.
2. Branch misprediction causes incorrectly fetched instructions to be discarded.
3. Data hazards occur when one instruction depends on the result of another instruction.
4. Pipeline stalls introduce unused clock cycles.
5. High memory latency causes the processor to wait for required data.
6. Limited execution resources may prevent instructions from being executed immediately.



Methods to Reduce CPI

1. Cache Memory

A suitable cache hierarchy reduces average memory-access time by keeping frequently used instructions and data closer to the processor.

2. Branch Prediction

Branch prediction estimates the outcome of a conditional branch so that instruction fetching can continue without waiting.

3. Instruction Pipelining

Pipelining allows multiple instructions to occupy different stages of execution simultaneously, increasing instruction throughput.

4. Out-of-Order Execution

Independent instructions can be executed while another instruction is waiting for data, improving processor resource utilization.

5. Hardware Prefetching

The processor can predict future memory requirements and fetch data before it is explicitly requested.

6. Superscalar Architecture

A superscalar processor can issue and execute multiple instructions during a single clock cycle when sufficient resources are available.

7. Efficient Instruction Scheduling

Rearranging independent instructions can reduce dependencies and minimize pipeline stalls.

Conclusion

The fetch-decode-execute cycle determines how instructions are processed, while CPI determines the average number of clock cycles required for each instruction. Cache memory, branch prediction, pipelining, prefetching, out-of-order execution, and superscalar architecture can reduce stalls and improve overall CPU performance.

QUESTION 3

The 8085 microprocessor provides different addressing modes to specify the location or source of operands. The choice of addressing mode determines how the processor obtains data and affects memory access, instruction length, and execution.

Addressing Modes of 8085

1. Immediate Addressing

In immediate addressing, the operand is directly included in the instruction.

Example:

MVI A, 25H

Here, 25H is directly loaded into the accumulator.

2. Register Addressing

In register addressing, the operand is stored in a CPU register.

Example:

MOV A, B

The contents of register B are transferred to register A.

3. Direct Addressing

In direct addressing, the memory address of the operand is explicitly specified in the instruction.

Example:

Data Transfer Instructions: MOV, MVI, LDA, STA

Arithmetic Instructions: ADD, SUB, INR, DCR

Logical Instructions: ANA, ORA, XRA, CMP

Branching Instructions: JMP, JZ, JNZ, JC

Stack Instructions: PUSH, POP

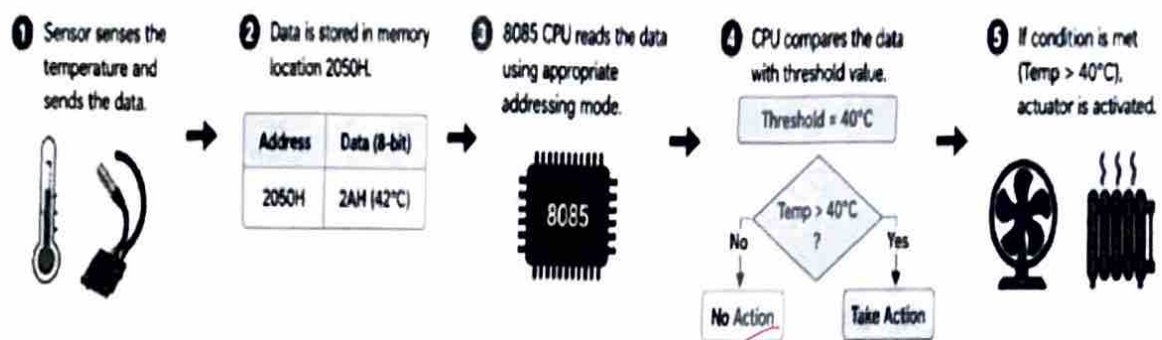
Control Instructions: NOP, HLT

Using the correct instruction for each operation reduces programming errors and improves program reliability.

Conclusion

Addressing modes determine how operands are accessed in the 8085 microprocessor. Proper selection of addressing modes, correct initialization of registers, accurate memory addressing, and appropriate use of the instruction set are essential for reliable and accurate program execution.

WORKING STEPS



QUESTION 4

The 8086 microprocessor uses segmented memory to address up to 1 MB of physical memory. Since the processor has 16-bit registers and a 20-bit address bus, segmentation is used to generate the required physical address.

Segment Registers

The major segment registers of the 8086 are:

1. CS – Code Segment
2. DS – Data Segment
3. SS – Stack Segment
4. ES – Extra Segment

Code Segment

The Code Segment contains program instructions.

1. CS stores the base address of the code segment.
2. IP contains the offset address of the next instruction.
3. The processor uses the CS:IP combination to fetch instructions.

Data Segment

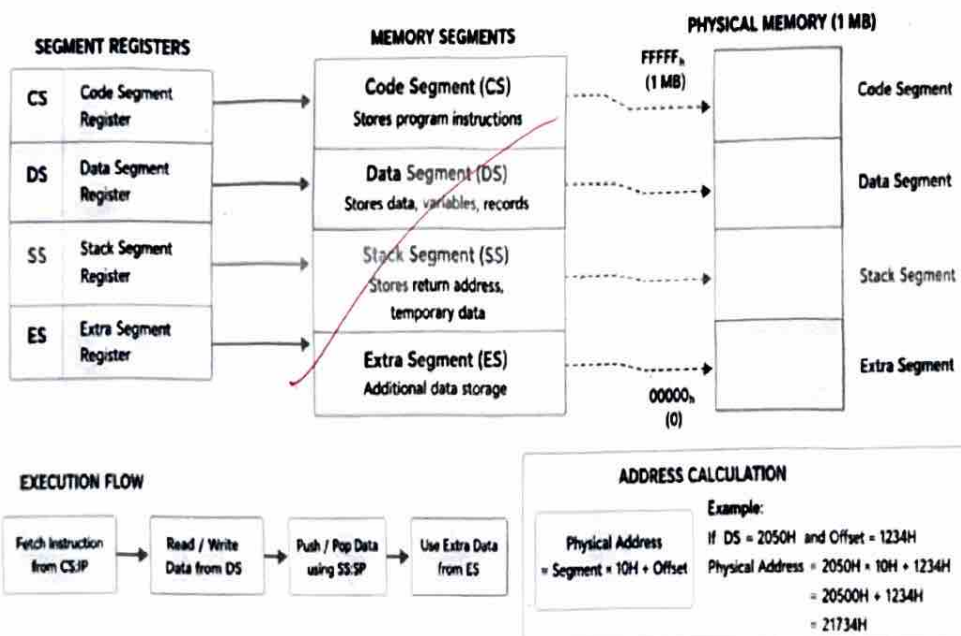
The Data Segment contains variables and other program data.

1. DS normally identifies the data segment.
2. An offset specifies the location of a particular data item.
3. Correct DS initialization is necessary for accessing the intended data.

Stack Segment

The Stack Segment is used for temporary storage, return addresses, register values, and procedure-related information.

1. SS identifies the stack segment.
2. SP identifies the current top of the stack.
3. PUSH stores data on the stack.
4. POP retrieves data from the stack.
5. CALL and RET instructions use the stack for procedure execution.



Extra Segment

The Extra Segment provides an additional data segment and is commonly used during string operations.

Physical Address Calculation

The physical address is 1000 is calculated using

$$\text{Physical Address} = \text{Segment Address} \times 16 + \text{Offset}$$

Example:

$$\text{Segment} = 2000$$

$$\text{Offset} = 0000$$

$$\text{Physical Address} = 2000 \times 16 + 0000$$

$$\text{Physical Address} = 32000$$

Problem Caused by Improper Segment Handling

1. An incorrect CS value can cause instructions to be fetched from an unintended memory location.
2. An incorrect IP value can cause instructions to be fetched from the wrong memory address.
3. An incorrect SI value can corrupt stack operations.
4. An incorrect DI value can result in corrupt data access.
5. Incorrect ESI/EEDI operations can corrupt pointers and data.
6. Unbalanced PUSH and POP operations can corrupt program execution.
7. Incorrect register manipulation can result in processing instructions incorrectly.
8. Improper stack management may corrupt the memory and affect further operations.

Instruction Execution - 8086

The general instruction execution process is

$$\text{Fetch} \rightarrow \text{Decode} \rightarrow \text{Operands Access} \rightarrow \text{Execute} \rightarrow \text{Store Result}$$

The 8086 instruction (8086) handles instruction fetching and its related operations, while the execution (8086) handles all arithmetic operations.

Instruction Modes of 8086

The instruction addressing modes include

1. Immediate addressing
2. Register addressing
3. Direct addressing
4. Register indirect addressing
5. Base addressing
6. Indexed addressing
7. Base indexed addressing

Correct addressing-mode selection ensures that the processor accesses the intended operand.

Conclusion

8086 segmentation organizes memory into code, data, stack, and extra segments. Correct initialization of segment registers, proper stack management, accurate instruction execution, and suitable addressing modes are necessary to prevent incorrect data access and stack-related errors.

QUESTION 5

Digital computers internally process information using binary numbers, whereas hexadecimal provides a compact and readable representation of binary values. Conversion between hexadecimal and binary is therefore important in low-level programming, debugging, memory inspection, and digital communication.

Relationship Between Hexadecimal and Binary

Each hexadecimal digit represents exactly four binary bits.

Importance of Number System Conversion

1. Hexadecimal provides a compact representation of binary information.
2. It makes memory addresses easier to read and interpret.
3. It simplifies inspection of machine-level data.
4. It is useful for analyzing instruction codes.
5. It reduces the difficulty of handling long sequences of binary digits.
6. The direct relationship between hexadecimal and four-bit binary groups makes conversion straightforward.

Effects of Incorrect Conversion

1. An incorrect hexadecimal digit changes the corresponding four binary bits.
2. Incorrect bits can produce a different numerical value.
3. If the value represents a memory address, the processor may access an unintended location.
4. If the value represents an instruction, the opcode or operand may change.
5. An incorrect operand can produce an incorrect arithmetic or logical result.
6. A changed control value can alter program execution.
7. Errors can propagate when an incorrect result is used by subsequent instructions.

Efficient CPU Design

1. Pipelining

Pipelining overlaps different stages of instruction execution and improves instruction throughput.

2. Cache Memory

2. Checksum
3. Cyclic Redundancy Check (CRC)
4. Automated conversion routines
5. Data validation
6. Test cases
7. Benchmark workloads

Conclusion

Hexadecimal provides a compact representation of binary information and is useful for debugging, memory inspection, and machine-level data analysis. Accurate conversion is important because an incorrect bit pattern can change data, memory addresses, or instructions. Efficient CPU architecture, error-detection mechanisms, and systematic benchmarking can improve both the speed and reliability of real-time computer systems.

